

JavaScript Async Parallel

[Back to JS page](#)

Table of Contents

- [JavaScript Async Parallel](#)
 - [Sequential Operations](#)
 - [Example 1](#)
 - [Independent Operations](#)
 - [Example 2](#)
 - [Promise.all\(\).](#)
 - [Example 3](#)
 - [Promise.allSettled\(\).](#)
 - [Example 4](#)
 - [Promise.race\(\).](#)
 - [Example 5](#)
 - [Document](#)
 - [Reference](#)

Sequential Operations

Sequential operations run one after another. Each operation waits for the previous one to complete:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Async Parallel</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Sequential Operations</h4>
<p id="demo"></p>

<script>
async function sequential() {
  let result1 = await new Promise(function(resolve) {
    setTimeout(function() {
      resolve("First result");
    }, 1000);
  });

  let result2 = await new Promise(function(resolve) {
    setTimeout(function() {
      resolve("Second result");
    }, 1000);
  });

  document.getElementById("demo").innerHTML = result1 + "<br>" + result2 + "<br>Total: ~2 second
}

sequential();
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Independent Operations

Independent operations do not depend on each other. They can run in parallel to save time:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Async Parallel</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Independent Operations</h4>
<p id="demo"></p>

<script>
async function independent() {
  let promise1 = new Promise(function(resolve) {
    setTimeout(function() {
      resolve("Task 1 done");
    }, 1000);
  });

  let promise2 = new Promise(function(resolve) {
    setTimeout(function() {
      resolve("Task 2 done");
    }, 1000);
  });

  // Start both promises, then await both
  let result1 = await promise1;
  let result2 = await promise2;

  document.getElementById("demo").innerHTML = result1 + "<br>" + result2 + "<br>Total: ~1 second
}

independent();
</script>

</body>
</html>

```

Example 2

Result [View Example](#)

Promise.all()

`Promise.all()` runs multiple promises in parallel and waits for ALL of them to complete:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Async Parallel</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Promise.all()</h4>
<p id="demo"></p>

<script>
const promise1 = new Promise(function(resolve) {
  setTimeout(function() { resolve("Promise 1 done"); }, 1000);
});

const promise2 = new Promise(function(resolve) {
  setTimeout(function() { resolve("Promise 2 done"); }, 2000);
});

const promise3 = new Promise(function(resolve) {
  setTimeout(function() { resolve("Promise 3 done"); }, 1500);
});

Promise.all([promise1, promise2, promise3]).then(function(values) {
  document.getElementById("demo").innerHTML =
    "All promises resolved in parallel:<br>" +
    values[0] + "<br>" +
    values[1] + "<br>" +
    values[2] + "<br>" +
    "Total time: ~2 seconds (not 4.5)";
});
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Promise.allSettled()

`Promise.allSettled()` waits for all promises to complete (regardless of resolve or reject):

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Async Parallel</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Promise.allSettled()</h4>
<p id="demo"></p>

<script>
const promise1 = new Promise(function(resolve) {
  setTimeout(function() { resolve("Success!"); }, 1000);
});

const promise2 = new Promise(function(resolve, reject) {
  setTimeout(function() { reject("Failed!"); }, 500);
});

const promise3 = new Promise(function(resolve) {
  setTimeout(function() { resolve("Done!"); }, 1500);
});

Promise.allSettled([promise1, promise2, promise3]).then(function(results) {
  let text = "";
  for (let result of results) {
    text += "Status: " + result.status + ", ";
    if (result.status === "fulfilled") {
      text += "Value: " + result.value;
    } else {
      text += "Reason: " + result.reason;
    }
    text += "<br>";
  }
  document.getElementById("demo").innerHTML = text;
});
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

Promise.race()

`Promise.race()` returns as soon as the FIRST promise settles (resolves or rejects):

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Async Parallel</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Promise.race()</h4>
<p id="demo"></p>

<script>
const promise1 = new Promise(function(resolve) {
  setTimeout(function() { resolve("Fast Promise won!"); }, 1000);
});

const promise2 = new Promise(function(resolve) {
  setTimeout(function() { resolve("Slow Promise"); }, 3000);
});

Promise.race([promise1, promise2]).then(function(value) {
  document.getElementById("demo").innerHTML =
    "Winner: " + value + "<br>" +
    "Promise.race() returns the fastest result";
});
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Document

Document in project

You can [Download PDF](#) file.

Reference

- [W3Schools JavaScript Async Parallel](#)